Arabian Journal for Science and Engineering
https://doi.org/10.1007/s13369-023-07971-x

RESEARCH ARTICLE-COMPUTER ENGINEERING AND COMPUTER SCIENCE

HAUOPM: High Average Utility Occupancy Pattern Mining

Mathe John Kenny Kumar[1] · **Dipti Rana**[1]

Received: 5 November 2022 / Accepted: 23 May 2023
© King Fahd University of Petroleum & Minerals 2023

Abstract
Considerable research has been dedicated to the area of data mining. Among the techniques used for this purpose, frequent pattern mining focuses on revealing patterns that occur frequently in the data. The field of data mining has expanded to include the extraction of patterns that can be beneficial to organizations, known as high-utility pattern mining. To improve the quality of these patterns, occupancy has been introduced as an interestingness measure in recent research. As a result, a new approach known as high-utility occupancy pattern mining (HUOPM) has been developed to identify patterns that make a significant contribution to their supporting transactions, as determined by a utility occupancy metric. A high-utility occupancy pattern is defined as a pattern whose utility occupancy exceeds a threshold value specified by the user. In the realm of transactional databases, the conventional definition of utility occupancy for an itemset involves adding up the utility occupancies for each of its supporting transactions. However, a key drawback of the traditional approach to HUOPM is that it does not account for the length of an itemset. Consequently, some items within an itemset may have higher utility occupancies than others, and the resulting representation may not accurately reflect the relative importance of each item within the itemset. To overcome this limitation, a novel *high average utility occupancy pattern mining* (HAUOPM) algorithm has been proposed. This algorithm uses a novel measure called *average utility occupancy* to mine *high average utility occupancy patterns*. As HAUOPM is not anti-monotonic, a novel *average utility occupancy upper bound* measure is used to prune unpromising itemsets. It can be observed that when two itemsets are compared, utility of one itemset might be greater than the other, whereas its average utility occupancy is less than the other itemset. Therefore, it is possible that some high-quality patterns are not discovered. To overcome this limitation, we introduce a novel measure called *transaction utility occupancy* to discover high-quality patterns. A novel *average utility occupancy tree* structure is used to enumerate the search space. This research uses a modified *average utility occupancy list* structure to store information about items/itemsets. Extensive experimentation has been performed on real and synthetic datasets to the test the effectiveness of the proposed HAUOPM algorithm.

Keywords Utility occupancy · Transaction utility occupancy · Average utility occupancy · Average utility occupancy list

1 Introduction

Data mining [1] has been a well-researched topic for many years, with a focus on techniques such as frequent pattern mining (FPM) [2, 3] and high-utility pattern mining (HUPM) [4–9]. FPM is used to extract patterns whose frequency is greater than the minimum support threshold. While FPM has many advantages such as identifying hidden relationships between variables and supporting decision-making, it also has some disadvantages. Some of them are:

1. High computational complexity: FPM requires a lot of computational resources, especially when dealing with large datasets. The process can be time-consuming and computationally expensive, which may limit its applicability to real-world problems.
2. Data sparsity: FPM may produce a large number of candidate patterns, many of which may be infrequent or irrelevant. This can lead to a high degree of data sparsity, which makes it difficult to extract meaningful insights from the data.
3. Lack of interpretability: FPM can produce a large number of complex patterns that are difficult to interpret. This

✉ Mathe John Kenny Kumar
ds18co001@coed.svnit.ac.in

Dipti Rana
dpr@coed.svnit.ac.in

[1] Computer Engineering, Sardar Vallabhbhai National Institute of Technology, Keval Chowk, Surat, Gujarat 395007, India

Published online: 21 June 2023

can make it difficult to understand the underlying relationships between variables and to use the patterns for decision-making.
4. FPM cannot compute patterns that are profitable to an organization.

On the other hand, HUPM is used to compute patterns that are considered profitable, i.e. those with a utility greater than the user-defined minimum threshold. This technique has several advantages over FPM. For example, a rare itemset such as a camera and laptop appearing only once in the database can be more profitable than a common itemset such as milk and cereals appearing 10 times. Major limitations of HUPM are as follows:

- Generating a large number of candidates is a common problem for traditional high-utility pattern mining (HUPM) algorithms, leading to extensive runtime and memory usage.
- Low-utility patterns, although potentially significant, may not be extracted by traditional high-utility pattern mining (HUPM) algorithms.
- Traditional HUPM algorithm do not consider other measures such as recency [10] and occupancy [11, 12].

The limitations of traditional high-utility pattern mining (HUPM) have led to the introduction of new measures such as occupancy [11], which can help to overcome these challenges. Occupancy, as a measure, was introduced to assess the contribution of a pattern in its supporting transactions. We propose a novel algorithm, called HAUOPM, which is designed to efficiently extract high average utility occupancy patterns, drawing inspiration from the previously proposed HAUI-Miner [13] algorithm.

1.1 Illustrative Comparison of HUOPM and the Proposed System

For example, consider a travel log containing routes of five customers across the state of Rajasthan, India. Table 1 contains travel logs of five customers. For example, customer1 travels the route $Delhi \rightarrow Agra \rightarrow Jaipur \rightarrow Mandawa \rightarrow Bikaner \rightarrow Jaisalmer$. For customer1, Delhi(200) represents the city Delhi and 200 represents the profit earned by the travel agency. According to HUOPM [14], *utility occupancy* of *{Delhi, Jaisalmer}* for customer1 is $(200 + 430)/(200 + 240 + 300 + 350 + 400 + 430) = 0.328$, where $200 + 430 = 630$ is the profit from the route *{Delhi, Jaisalmer}* and $240 + 300 + 350 + 430 = 1920$ is the total profit earned from customer1. *Utility occupancy* of *{Delhi, Jaisalmer}* for the whole travel log is $0.328/1 = 0.328$, where 1 represents the number of times *{Delhi, Jaisalmer}* occurs in the whole travel log. For example, *utility occupancy* of *{Jodhpur}* for customer4 can be calculated as $100/100 = 1.0$. As *{Jodhpur}* occurs only once in the travel log, the total *utility occupancy* of *{jodhpur}* is $1.0/1 = 1.0$. Moreover, HUOPM algorithm [14] does not consider the length of the itemset while computing the *utility occupancy* of an itemset. For example, *utility occupancy* of *{Delhi, Jaisalmer}* in the travel log is 0.328. The value 0.328 does not give a fair representation of the *utility occupancy* of *{Delhi}*, *{Jaisalmer}*. Therefore, a novel measure called *average utility occupancy* is introduced. *Average utility occupancy* of *{Delhi, jaisalmer}* can be calculated as $0.328/2 = 0.164$, where 0.164 represents the *average utility occupancy* of *{Delhi, jaisalmer}* and 2 represents the number of items in the itemset. Similarly *average utility occupancy* of *{johdhpur}* can be calculated as $1.0/1 = 1.0$. It can be observed that the *average utility occupancy* of $\{Delhi, Jaisalmer\} < \{jodhpur\}$, whereas the utility of $\{Delhi, Jaisalmer\} > \{jodhpur\}$. Therefore, high-quality patterns would not be mined if only *utility occupancy* measure is taken into consideration and not the utility of an itemset. To overcome this limitation, this research introduces the concept of *transaction utility occupancy*. For example, total utility of all the customers, i.e. customer1, customer2, cutormer3, customer4, customer5, is 3150. As *{Delhi, Jaisalmer}* occurs only once in the travel log in customer1, *average transaction utility occupancy* of *{Delhi, jaisalmer}* is $(0.164) * (1920/3150) = 0.099$, where 1920 is the *transaction utility occupancy* of *{Delhi, jaisalmer}*. Similarly, *average transaction utility occupancy* of *{jodhpur}* is $(1.0) * (100/3150) = 0.031$, where 100 is the total utility of customer4. As $0.099 > 0.031$ the sub-route/pattern *{Delhi, Jaisalmer}* is recommended to other customers instead of *{jodhpur}*. *Utility occupancy* is applicable in real-life applications such as web page recommendation [11], market basket analysis [15–17], mobile service provider [12].

1.2 Contributions

This research has the following contributions.

- *High average utility occupancy pattern mining* (HAUOPM) is the first work to propose *average utility occupancy (AUO)* as a measure to mine high average utility occupancy patterns (HAUOPs).
- A novel measure called *transaction utility occupancy (tuo)* is used to mine high-quality patterns.
- A novel pruning technique called *average utility occupancy upper bound* (auoub) is used to eliminate unpromising itemsets.
- A novel tree structure called average utility occupancy tree (AUO-Tree) is used to enumerate the search space.

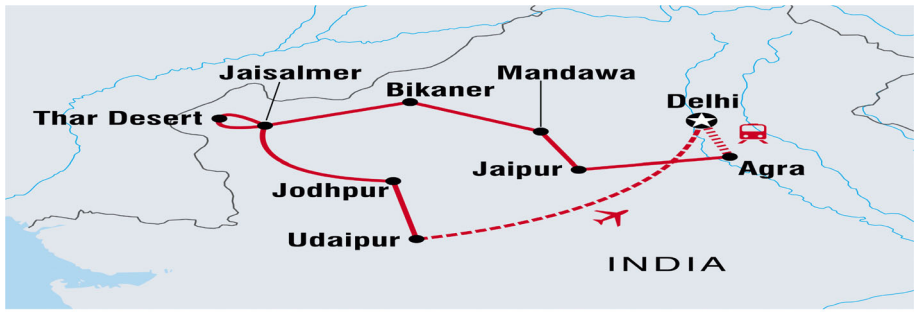


Fig. 1 Rajasthan-trip

Table 1 {Travel Log}

$cust_{id}$	Travel Log					
Customer1	Delhi (200)	Agra (240)	Jaipur (300)	Mandawa (350)	Bikaner (400)	Jaisalmer (430)
Customer2	Delhi (120)	Agra (200)				
Customer3	Delhi (100)					
Customer4	Jodhpur (100)					
Customer5	Delhi (110)	Udaipur (200)	Thar Desert (400)			

- A modified list structure called average utility occupancy list (AUOL) is used to store information about item/itemsets.
- Extensive experiments have been performed on real and synthetic datasets to test the efficiency of the proposed *high average utility occupancy pattern mining* (HAUOPM) algorithm.

1.3 Organization of the Paper

This research is structured as follows. Section 2 describes various algorithms based on occupancy pattern mining. Section 3 lists various definitions used throughout this research. Section 4 lists three proposed algorithms for mining high average utility occupancy patterns. Section 5 is used to test the proposed HAUOPM algorithm using various real and synthetic datasets. Section 6 explains in brief about the work done and potential applications of the proposed system.

2 Related Work

2.1 Occupancy-Based Pattern Mining: A Critical Analysis

In several high-utility pattern mining (HUPM) algorithms, the patterns that have a significant contribution to the utility of their supporting transactions may not be effectively extracted. Previously, DOFIA [11] algorithm used *support* and *average occupancy* as measures for high-quality pattern recommendation. The limitation of this approach is that it does not take the length of the itemsets into consideration. Moreover, some itemsets may have high utility, but their occupancies might be lower and vice versa. This results in missing out of important itemsets. DOFRA [18] algorithm was designed for high-quality sequence pattern recommendation. Both DOFIA [11] and DOFRA [18] algorithms were based on incorporating the occupancy measure into frequent pattern mining. Both the approaches do not take the length of the itemset into consideration. As a consequence, it was not feasible to obtain a balanced and accurate representation of the occupancy of each individual item within an itemset. The OCEAN [12] algorithm used *utility occupancy* as a measure to mine high occupancy patterns. Utility occupancy measures the contribution of a pattern within its supporting transactions. The limitation of OCEAN [12] algorithm is that it could not mine the complete set of high-utility occupancy patterns. To overcome this limitation, high-utility occupancy pattern mining (HUOPM) [14] was proposed. Both OCEAN [12] and HUOPM [14] algorithm fail to consider the length of the itemset. UHUOPM [19] was proposed to mine high-utility patterns from uncertain databases. UHUOPM [19] algorithm uses the *utility occupancy* as a measure to discover the complete set of high-utility occupancy patterns from uncertain databases. Similar to OCEAN [12] and HUOPM [14], UHUOPM [19] algorithm also suffers from the same set of limitations. In 2022, HUOMI [20] algorithm was introduced, which presented a new approach for real-time data processing in dynamic environments, allowing for efficient analysis of large datasets. One of its notable strengths is its ability to handle expanding databases with ease. The algorithm also proposed an optimized matrix-based data structure and enhanced pruning strategy to effectively deal with the dynamic environment. A paper published in 2022 introduced HOMI [21] (High occupancy pattern mining on incremental databases), which utilizes a depth-first search approach

Table 2 Occupancy-based pattern mining

Algorithm	Explanation	Measure	Data structure
DOFIA [11]	Explores the upper-bound properties on occupancy to reduce the search space	Occupancy	Tree based
DOFRA [18]	Algorithm is used for high quality sequential pattern recommendation	Occupancy	Prefix projected database tree
OCEAN [12]	Algorithm derives an upper-bound of occupancy based on utility	Utility occupancy	List based
HUOPM [14]	Algorithm mines the complete set of HUOPs	utility occupancy	List based
UHUOPM [19]	Algorithm mines HUOPs from uncertain databases	Utility occupancy	List based
HOITMO [23]	Algorithm mine high occupancy patterns using transaction occupancy measure	Transaction occupancy	List based
HUOMI [20]	Algorithm mine high occupancy patterns from increasing databases	Occupancy based utility	Matrix or grid based
HOMI [21]	Algorithm mine high occupancy patterns from incremental databases	Occupancy	List based
FHOI [22]	Algorithm applies novel conditions to eliminate unqualified itemsets to generate k-items	Occupancy	Equivalence class tree(ECT) and high occupancy itemset tree (HOIT)

to identify and extract itemsets from incremental databases. This method is an improvement over previous literature, as most existing algorithms rely on the breadth-first search approach, which often leads to significant memory consumption. FHOI [22] applies additional conditions to eliminate unqualified itemsets and integrate the property of equivalence class and reduces the runtime to generate k-itemsets. Secondly, the authors used an equivalence class tree (ECT) to efficiently compute the occupancy of itemsets. Thirdly, the authors used a high occupancy itemset tree (HOIT) to store high occupancy itemsets found during the mining process. HOITMO [23] introduced the concept of *transaction occupancy* and *itemset occupancy* but it failed to address the problem of itemset length. HAUI-Miner [13] proposed an efficient algorithm to mine High Average Utility Itemsets. [24] presents a detailed review about all the High Average Utility Itemset mining algorithms. Inspired by HAUI-Miner [13] we propose a novel high average utility occupancy pattern mining (HAUOPM) algorithm to mine high average utility occupancy patterns from transactional databases. Comprehensive summary of the related work is presented in Table 2. Critical analysis and a comparison to justify the proposed system is presented in Table 3.

Table 3 Critical analysis: occupancy-based pattern mining algorithms

Algorithm	Mining using itemset length	Mining quality patterns of high utility
DOFIA [11]	No	No
DOFRA [18]	No	No
OCEAN [12]	No	No
HUOPM [14]	No	No
UHUOPM [19]	No	No
HOITMO [23]	No	No
HUOMI [20]	No	No
HOMI [21]	No	No
FHOI [22]	No	No
HAUOPM (PROPOSED)	Yes	Yes

3 Definitions and Problem Statement

This section lists the various preliminaries related to high average utility occupancy pattern mining (HAUOPM) and uses the transaction database Table 4 and external utility table 5 to define and illustrate various notations related to HAUOPM.

3.1 Definitions

Consider $IT = \{it_1, it_2, \ldots, it_m\}$ be a group of m unique items and let DB be a quantitative database, where $DB = \{Tr_1, Tr_2, ...Tr_n\}$. Every transaction $Tr_d \in DB$ and $1 \leq d \leq n$ where d is a unique identifier for each transaction represented by Tr_d. Each item $it_p (1 \leq p \leq m)$ contains the quantity purchased which is represented by internal utility. Internal utility of an item it_p in a transaction Tr_d is denoted by $iu(it_p, Tr_d)$. External utility of an item is denoted by $eu(it_p)$.

Table 4 Transaction database

Tr_d	Items with quantities
Tr_1	a:1,b:1,e:2
Tr_2	b:2,e:1
Tr_3	c:1
Tr_4	a:2,d:1
Tr_5	a:1,d:3
Tr_6	b:2,d:1
Tr_7	c:1
Tr_8	a:1,b:3
Tr_9	a:2,b:2,d:1
Tr_{10}	d:2,e:2

Table 5 External utilities

Item	Profit
a	2
b	3
c	1
d	3
e	2

Item utility for transaction Tr_d can be defined as, $ut(it_p, Tr_d) = iu(it_p, Tr_d) \times eu(it_p)$. For example, from Tables 4 and 5, $iu(a, Tr_1) = 1$, $eu(a) = 2$, $ut(a, Tr_1) = 1 \times 2 = 2$. A set of k unique items $X = \{it_1, it_2, ..., it_k\}$ where $X \subseteq IT$, is called a k-itemset where k is the length of the itemset X, $X \subseteq Tr_d$.

Definition 3.1 (*Utility of an itemset in a transaction* [25]) Utility of an itemset X, in a transaction Tr_d, is denoted as $ut(X, Tr_d)$, and is defined as,

$$ut(X, Tr_d) = \sum_{it_p \in X \wedge X \subseteq Tr_d} ut(it_p, Tr_d). \tag{1}$$

For example, using Tables 4 and 5, $ut(ab, Tr_1) = 1 \times 2 + 1 \times 3 = 5$.

Definition 3.2 (*Utility of an itemset in a database* [25]) Utility of an itemset X, in a transactional database DB, is denoted as $ut(X)$, and is defined as,

$$ut(X) = \sum_{X \subseteq Tr_d \wedge Tr_d \in DB} ut(X, Tr_d). \tag{2}$$

For example, using Tables 4 and 5, $ut(ab) = ut(ab, Tr_1) + ut(ab, Tr_8) + ut(ab, Tr_9) = 5 + 11 + 10 = 26$.

Definition 3.3 (*Transaction utility* [25]) Transaction utility tu, of a transaction Tr_d, is denoted as $tu(Tr_d)$, and is defined as,

$$tu(Tr_d) = \sum_{it_p \in Tr_d \wedge Tr_d \in DB} ut(it_p, Tr_d). \tag{3}$$

For example, from Tables 4 and 5, $tu(Tr_1) = ut(a, Tr_1) + ut(b, Tr_1) + ut(e, Tr_1) = 2 + 3 + 4 = 9$.

Definition 3.4 (*Database utility* [25]) The sum of transaction utilities of all the transactions in a database DB, where $Tr_d \in DB$, is denoted as $ut(DB)$, and is defined as,

$$ut(DB) = \sum_{Tr_d \in DB} tu(Tr_d). \tag{4}$$

For example, from Tables 4 and 5, $ut(DB) = 80$.

Definition 3.5 (*Utility occupancy of an itemset in a transaction* [14]) Utility occupancy of an itemset X, in a supporting transaction Tr_d, is denoted by $uo(X, Tr_d)$, and is defined as the ratio of the utility of X divided by the transaction utility tu of the supporting transaction Tr_d.

$$uo(X, Tr_d) = ut(X, Tr_d)/tu(Tr_d). \tag{5}$$

where $X \subseteq Tr_d$ and $Tr_d \in DB$.
For example, Since $tu(Tr_1) = 9$, $tu(Tr_8) = 11$ and $tu(Tr_9) = 13$, $uo(ab, Tr_1) = 5/9 = 0.55$, $uo(ab, Tr_8) = 11/11 = 1$ and $uo(ab, Tr_9) = 10/13 = 0.76$.

Definition 3.6 (*SupportCount* [15]) Supportcount is the number of times an itemset X occurs in a transactional database DB. It is denoted by $supportcount(X)$ and is defined as,

$$supportcount(X) = Count(X, DB). \tag{6}$$

where $X \subseteq Tr_d$ and $Tr_d \in DB$. **For example**, from Tables 4 and 5, $supportcount(ab) = 3$, $supportcount(c) = 2$.

Property 1 (*SupportCount* [15]) The $supportcount$ of an itemset X, in a transactional database DB is:

1. Not less than the $supportcount$ of its subsets (and),
2. Not greater than the $supportcount$ of its supersets.

Definition 3.7 (*Utility occupancy of an itemset in a transactional database* [14]) The utility-occupancy of an itemset X, in a transactional database DB, is denoted as $uo(X)$, and is defined as,

$$uo(X) = \frac{\sum_{X \subseteq Tr_d \wedge Tr_d \in DB} uo(X, Tr_d)}{supportcount(X)}. \tag{7}$$

For example, $uo(ab) = (uo(ab, Tr_1) + uo(ab, Tr_8) + uo(ab, Tr_9))/supportcount(ab) = (0.55 + 1 + 0.76)/3 = 0.77$.

Definition 3.8 (*Length of an itemset*) The length of an itemset X is denoted by $len(X)$ and can be defined as the total number of items it_p in X.

$$len(X) = count(it_p, X). \tag{8}$$

where $it_p \in X$ and $X \subseteq Tr_d$. **For example**, $len(ab) = 2$.

Utility occupancy of an itemset in a transactional database does not truly represent the occupancy of each item of an itemset in its supporting transactions. Inspired by "average utility" measure of HAUI-Miner[13], a novel "average utility occupancy" measure is defined.

Definition 3.9 (*Average utility occupancy of an itemset in a transactional database*) The average utility occupancy of an itemset X, in a transactional database DB, is denoted as $auo(X)$ and is defined as,

$$auo(X) = uo(X)/len(X). \tag{9}$$

where $X \subseteq Tr_d$ and $Tr_d \in DB$. **For example**, $auo(ab) = uo(ab)/len(ab) = 0.77/2 = 0.39$. **For example**, $auo(c) = uo(c)/len(c) = 1/1 = 1$.

Definition 3.10 (*Sum of transaction utilities of supporting transactions*) Sum of transaction utilities of supporting transactions is denoted by $stu(X)$ and can be defined as,

$$stu(X) = \sum_{X \subseteq Tr_d \wedge Tr_d \in DB} tu(Tr_d). \tag{10}$$

For example, $stu(ab) = Tr_1 + Tr_8 + Tr_9 = 9 + 11 + 13 = 33$.

From Tables 4 and 5, $auo(c) = 1.0$ and $ut(c) = 2$. Similarly, $auo(ab) = 0.39$ and $ut(ab) = 26$. It can be clearly observed that even though $auo(c) > auo(ab)$, $ut(c) < ut(ab)$, utility occupancy as a sole measure fails to mine the complete set of high-quality patterns. Previously, the concept of transaction occupancy was introduced in [23]. In [23], transaction occupancy was introduced as a factor along with frequency to mine high occupancy patterns. In this work, we introduce a new measure called *transaction utility occupancy* (tuo) which uses utility as a factor to calculate the transaction occupancy of an itemset in its supporting transactions.

Definition 3.11 (*Transaction utility occupancy*) Transaction utility occupancy of an itemset X, in a transactional database DB, is denoted as $tuo(X)$, and can be defined as,

$$tuo(X) = stu(X)/ut(DB). \tag{11}$$

where $X \subseteq Tr_d$ and $Tr_d \in DB$. **For example**, $tuo(ab) = 33/80 = 0.41$.

Definition 3.12 (*Average transaction utility occupancy*) Average transaction utility occupancy of an itemset X, in a transactional database DB, is denoted as $atuo(X)$ and can be defined as,

$$atuo(X) = auo(X) * tuo(X). \tag{12}$$

where $X \subseteq Tr_d$ and $Tr_d \in DB$. **For example**, $atuo(ab) = 0.39 * .41 = 0.16$. $atuo(c) = 1 * .02 = 0.02$. As $atuo(ab) > atuo(c)$, the itemset $\{ab\}$ is said to be a higher quality itemset than $\{c\}$ even though $auo(c) > auo(ab)$.

Minimum support threshold is denoted by min_sup, and minimum occupancy threshold is denoted by min_occ.

Definition 3.13 User defined minimum occupancy threshold is denoted by min_occ and is defined as,

$$0 \leq min_occ \leq 1 \tag{13}$$

For example, we assume $min_occ = 0.37$.

Definition 3.14 (*High Average Utility Occupancy Pattern*) An itemset or a pattern is said to be a high average utility occupancy pattern (HAUOP) if its $supportcount(X) \geq min_sup$ and, $atuo(X) \geq min_occ$. **For example**, considering $min_sup = 2$ and $min_occ = 0.37$, $supportcount(b) = 5.0$ and $atuo(b) = 0.38$, therefore item {b} is a high average utility occupancy pattern (HAUOP).

4 Proposed HAUOPM Algorithm

In this section, we propose a novel high average utility occupancy pattern mining (HAUOPM) algorithm to mine all the high average utility occupancy patterns (HAUOPs). The average utility occupancy lists (AUOLs) of all extensions of each item $it_p \in IT$ are recursively processed using a depth-first method to mine all the HAUOPs. A modified average utility occupancy list (AUOL) structure is used to store occupancy and maximum utility occupancy values. A novel upper-bound called average utility occupancy upper bound (auoub) is also used to prune unpromising itemsets. Algorithms 1, 2, 3 are explained with an illustrative example.

Step 1: The transactional database DB is scanned to calculate the $supportcount$ of all items $it_p \in IT$, definition 3.6, [15], [Algorithm 1 step 3]. **For example**, using Tables 4 and 5, the $supportcount$ of items are, $\{a = 5, b = 5, c = 2, d = 5, e = 3\}$.

Throughout the illustrative example, minimum support threshold *min_sup* is considered as 2 and minimum occupancy threshold *min_occ* is considered as 0.37.

Algorithm 1 HAUOPM

1: Input: DB, min_sup, min_occ
2: Output: Set of all $HAUOPs$
3: Scan DB to calculate the $supportcount$ of each item $it_p \in IT$ and tu of each transaction Tr_d;
4: Find $IT(supportcount(it_p) \geq min_sup)$;
5: Sort IT according to ascending order of $supportcount$ values;
6: Revise DB according to ascending order of $supportcount$ values;
7: Scan DB to construct $it_p.AUOL$ of each item $it_p \in IT$;
8: search-HAUOPM($\phi, IT, min_sup, min_occ$).
9: return HAUOPs

Algorithm 2 Search-HAUOPM

1: Input: $X, extensionsOfX, min_sup, min_occ$
2: Output: Set of all HAUOPs
3: $sumTopK \leftarrow 0$, $X_a.auoub \leftarrow 0$, $V_{avgocc} \leftarrow \emptyset$;
4: **for** $itemset\ X_a \in extensionsOfX$ **do**
5: calculate the values of $X_a.sup$, $X_a.avgituo$ from $X_a.AUOL$;
6: **if** $X_a.sup < min_sup$ **then**
7: $continue$;
8: **end if**
9: **if** $X_a.sup \geq min_sup \wedge X_a.avgituo \geq min_occ$ **then**
10: $HAUOPs \leftarrow HAUOPs \cup X_a$;
11: **end if**
12: calculate $X_a.imuo$ for each tuple of $X_a.AUOL$;
13: store the values in a vector V_{avgocc};
14: sort V_{avgocc} in descending order as $V_{\downarrow avgocc}$;
15: **for** $k \leftarrow 1$ to min_sup in $V_{\downarrow avgocc}$ **do**
16: $sumTopK \leftarrow sumTopK + V_{\downarrow avgocc}$;
17: **end for**
18: $X_a.auoub = [sumTopK/min_sup]$.
19: **if** $X_a.auoub \geq min_occ$ **then**
20: $extensionsOfX_a \leftarrow \emptyset$;
21: **for** $X_b \in extensionsOfX$ such that $a \prec b$ **do**
22: $X_{ab}.AUOL \leftarrow construct(X, X_a, X_b)$;
23: **end for**
24: **end if**
25: Search-HAUOPM($X_a, extensionsOfX_a, min_sup, min_occ$)
26: **end for**

Algorithm 3 Construct Procedure-AUOL

1: Input: X: An itemset, X_a:Extension of X with an item a, X_b:Extension of X with an item b
2: Output:$X_{ab}.AUOL$: AUOL of an itemset X_{ab}
3: set $X_{ab}.AUOL \leftarrow \emptyset$;
4: **for** $element\ E_a \in X_a.AUOL$ **do**
5: **if** $\exists E_a \in X_b.AUOL \wedge E_a.tid = E_b.tid$ **then**
6: **if** $X.AUOL \neq \emptyset$ **then**
7: Search $E \in X.AUOL, E.tid = E_a.tid$;
8: $E_b.imuo = max(E.imuo, E_a.imuo, E_b.imuo)$;
9: $E_{ab} \leftarrow (E_a.tid, E_a.iuo + E_b.iuo - E.iuo, E_b.imuo, tu)$;
10: **else**
11: $E_b.imuo = max(E_a.imuo, E_b.imuo)$;
12: $E_{ab} \leftarrow (E_a.tid, E_a.iuo + E_b.iuo, E_b.imuo, tu)$;
13: **end if**
14: $X_{ab}.AUOL \leftarrow X_{ab}.AUOL \cup E_{ab}$;
15: **end if**
16: **if** $X_{ab}.sup == \emptyset$ **then**
17: return $null$;
18: **end if**
19: **end for**

Theorem 1 (Search Space) *The complete search space to mine HAUOPs can be represented using an AUO-Tree [26] where items are sorted in ascending order of their $supportcount$.*

Proof From the AUO-Tree [26], $2^m - 1$ itemsets can be formed from the items in IT. Let m be the number of distinct items in IT. Using a depth first search approach, all the supersets of items in IT can be enumerated according to their $supportcount$. This representation is complete and correct as all the supersets of the root node are explored.

For example, items(nodes) are arranged according to ascending order of $supportcount$ values: $c \prec e \prec a \prec b \prec d$. The number of items(nodes) and itemsets(nodes) in the search space is $2^5 - 1 = 31$. The complete search space is represented in Fig. 2.

Theorem 2 (Global Downward Closure Property of AUO-Tree) *In the proposed AUO-Tree, if the $supportcount$ of an item(node) or an itemset(node) is greater than min_sup, the $supportcount$ of its parent item(node) or itemset(node) is also greater than its min_sup. Let X^k be a k-itemset and X^{k-1} be its parent itemset, then $supportcount(X^k) \leq supportcount(X^{k-1})$ holds.*

Proof According to Apriori property [15], $supportcount(X^k) \leq supportcount(X^{k-1})$, therefore in the AUO-Tree, the global downward closure property holds. □

Pruning Strategy 1 [15] : The $supportcount$ of an item(node) it_p or an itemset(node) X is both monotonic as well as anti-monotonic. If the $supportcount$ of an item(node) or an itemset(node) is less than min_sup, the item/itemset can be pruned and none of its supersets are explored, definition 3.6, theorem 2, property 1, [Algorithm 1 Step 4].

For example, $supportcount$ of all items using Tables 4 and 5 are $\{a = 5, b = 5, c = 2, d = 5, e = 3\}$. As the $supportcount$ of all the items(nodes) is no less than the min_sup, no item is pruned.

Step 2: All items(nodes) $it_p \in IT$ are sorted in ascending order of their $supportcount$ values whose $supportcount \geq min_sup$, Theorem 1, property 1, [Algorithm 1 Step 5].

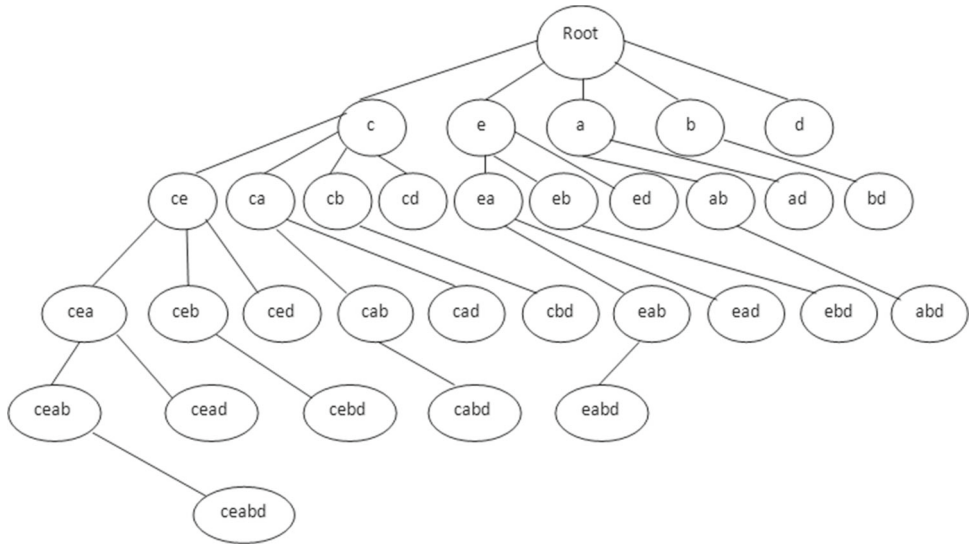
For example, items(nodes) are stored in the order, $c \prec e \prec a \prec b \prec d$.

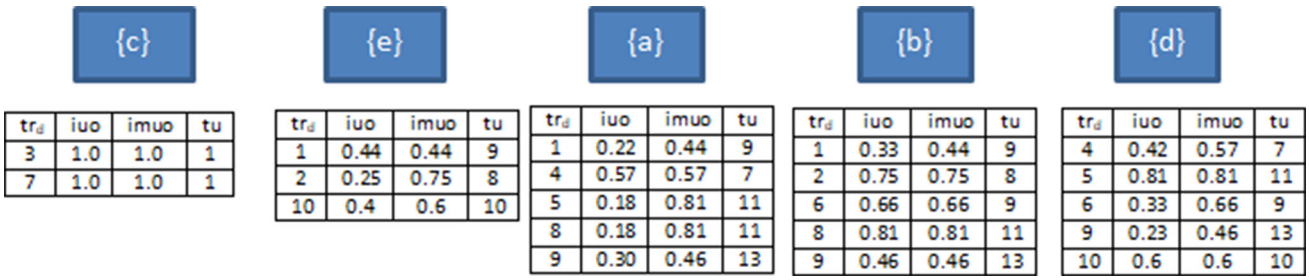
Step 3: The transactional database DB is revised for all items $it_p \in IT$, according to ascending order of $supportcount$ values, [Algorithm 1 Step 6].

For example, the transaction database DB is revised as Table 6.

Definition 4.1 (*Average Utility Occupancy List (AUOL) of an item/itemset*) Each node of the AUO-Tree is represented by an average utility occupancy list (AUOL) such that each

Fig. 2 Search-space





{c}

tr_d	iuo	imuo	tu
3	1.0	1.0	1
7	1.0	1.0	1

{e}

tr_d	iuo	imuo	tu
1	0.44	0.44	9
2	0.25	0.75	8
10	0.4	0.6	10

{a}

tr_d	iuo	imuo	tu
1	0.22	0.44	9
4	0.57	0.57	7
5	0.18	0.81	11
8	0.18	0.81	11
9	0.30	0.46	13

{b}

tr_d	iuo	imuo	tu
1	0.33	0.44	9
2	0.75	0.75	8
6	0.66	0.66	9
8	0.81	0.81	11
9	0.46	0.46	13

{d}

tr_d	iuo	imuo	tu
4	0.42	0.57	7
5	0.81	0.81	11
6	0.33	0.66	9
9	0.23	0.46	13
10	0.6	0.6	10

Fig. 3 AUOL of {c}, {e}, {a}, {b}, {d}

Table 6 Revised transaction database

Tr_d	Items with quantities
Tr_1	e:2,a:1,b:1
Tr_2	e:1,b:2
Tr_3	c:1
Tr_4	a:2,d:1
Tr_5	a:1,d:3
Tr_6	b:2,d:1
Tr_7	c:1
Tr_8	a:1,b:3
Tr_9	a:2,b:2,d:1
Tr_{10}	e:2,d:2

row contains $(Tr_d, iuo, imuo, tu)$ for each transaction Tr_d, containing $X.iuo$ [14], where $X.iuo$ represents the utility occupancy for each row of an item/itemset $X \subseteq Tr_d$ and is defined as $uo(X, Tr_d)$. $imuo$ represents the maximum utility occupancy for each row of an item $X \subseteq Tr_d$ and is defined as $max(iuo(X, Tr_d))$, where $X \subseteq Tr_d \wedge Tr_d \in DB$.
For example, the AUOL of $\{e\}$ is $\{(Tr_1, 0.44, 0.44, 9)(Tr_2, 0.25, 0.75, 8)(Tr_{10}, 0.4, 0.6, 10)\}$.

Step 4: The transactional database DB is scanned again to construct the average utility occupancy lists (AUOLs) of all items, $it_p \in IT$, definition 4.1, [Algorithm 1 Step 7].
For example, using Tables 6 and 5, AUOLs of {c}, {e}, {a}, {b}, {d} are constructed.

Definition 4.2 (*Support count of an item/itemset in AUOL*) Given AUOL, the supportcount of an itemset X is denoted by $X.sup$, and is defined as,

$$X.sup = count(tr_d, X.AUOL). \tag{14}$$

where $X \subseteq Tr_d$ and $Tr_d \in DB$. **For example**, $a.sup = 5$.

Definition 4.3 (*Utility occupancy of an item/itemset in AUOL* [14]) Given AUOL, the sum of utility occupancy of an itemset X is denoted by $X.sumiuo$ and is defined as,

$$X.sumiuo = \frac{\sum_{X \subseteq Tr_d \wedge Tr_d \in DB} X.iuo}{X.sup}. \tag{15}$$

For example, $a.sumiuo = 0.22 + 0.57 + 0.18 + 0.18 + 0.30/5 = 1.45/5 = 0.29$.

Definition 4.4 (*Average utility occupancy of an item/itemset in AUOL*) Given AUOL, the average utility occupancy of an item/itemset X is denoted by $X.avgiuo$ and is defined as,

$$X.avgiuo = X.sumiuo/len(X). \tag{16}$$

where $X \subseteq Tr_d$ and $Tr_d \in DB$. **For example**, $a.avgiuo = 0.29/1 = 0.29$.

Definition 4.5 (*Transaction utility occupancy of an item/itemset in AUOL*) Given AUOL, the transaction utility occupancy of an itemset X is denoted by $X.tuo$ and is defined as,

$$X.tuo = \frac{\sum_{X \subseteq Tr_d \wedge Tr_d \in DB} tu(Tr_d)}{tu(DB)}. \tag{17}$$

For example, $a.tuo = 51/80 = 0.63$.

Definition 4.6 (*Average transaction utility occupancy of an item/itemset in AUOL*) Given AUOL, the average transaction utility occupancy of an itemset X is denoted by $X.avgituo$ and is defined as,

$$X.avgituo = X.avgiuo * X.tuo. \tag{18}$$

where $X \subseteq Tr_D$ and $Tr_d \in DB$. **For example**, $a.avgituo = 0.29 * 0.63 = 0.18$.

Step 5: The search algorithm 2 uses a prefix based recursive approach to mine all the high average utility occupancy patterns (HAUOPs) using an average utility occupancy-tree (AUO-Tree) which is based on the set-enumeration [26] tree. A total of $2^m - 1$ non-empty itemsets can be formed by items in IT containing m elements. It uses a depth first approach to mine all HAUOPs.

Step 6: Consider item(node) {a} as a prefix. $a.sup = 5$, $a.avgituo = 0.18$. As $a.sup = 5 > 2(min_sup)$ but $a.avgituo = 0.18 < 0.37(min_occ)$, item {a} is not a HAUOP, [Algorithm 2 Steps 6-11].

Property 2 (*Average utility occupancy* [14]) The downward closure property for average utility occupancy does not hold. The auo of an itemset X, in a transactional database DB, is:

1. Not less than the auo of its subsets (or)
2. Not greater than the auo of its subsets.

4.1 Proposed Average Utility Occupancy Upper Bound (Auoub)

The supporting transactions of an itemset $X.AUOL$ are denoted by $transsup_X$. The supporting transactions of any possible high average utility occupancy pattern W which extends X are unknown, but it can be observed that $transsup_W \subseteq transsup_X$. HUOPM [14] established an upper-bound to mine high utility occupancy patterns. The upper-bound used in HUOPM [14] cannot be used in the proposed HAUOPM algorithm; inspired by OCEAN [12], HUOPM [14], we develop a theorem to establish an upper-bound to prune unpromising itemsets(nodes).

Lemma 1 *Let X be an itemset(node), and let $transsup_X$ represent the supporting transactions of X. For any itemset W, where W represents the extension of X, the upper bound on the average utility occupancy of W is:*

$$upper - bound(W) = \left[\frac{\sum_{topmin_sup, Tr_d \in transsup_X} X.imuo \downarrow}{min_sup}\right]. \tag{19}$$

Proof The value of $X.imuo$ is calculated for all transactions in $transsup_X$. The $X.imuo$ values are stored in a vector set which are sorted in descending order of their itemset maximum utility occupancy (imuo) values. The sorted vector set is denoted by $V_{\downarrow avgocc}$. As the average of top $k(0 < k < supportcount(W))$ is an upper-bound of the average of total $supportcount(W)$ values of $[\frac{\sum_{W \subseteq Tr_d \wedge Tr_d \in DB} X.imuo}{supportcount(W)}$. A high average utility occupancy pattern should have $supportcount$ equal to or greater than min_sup. Therefore, we have $min_sup \leq k \leq supportcount(W) \leq supportcount(X)$. Therefore,

$$\begin{aligned} auo(W) &\leq \left[\frac{\sum_{W \subseteq Tr_d \wedge Tr_d \in DB} X.imuo}{supportcount(W)}\right] \\ &\leq \left[\frac{\sum_{topk, Tr_d \in transsup_X} X.imuo \downarrow}{supportcount(W)}\right] \\ &\leq \left[\frac{\sum_{min_sup, Tr_d \in transsup_X} X.imuo \downarrow}{min_sup(X)}\right] \\ &\leq upper - bound(W). \end{aligned}$$

□

Therefore, for any itemset(node) X, $upper - bound(W)$ for average utility occupancy can be calculated, given minimum support, min_sup.

Theorem 3 (Partial downward closure property of AUO-Tree) *Let X^k be a $k - itemset$ and let its parent node be denoted by X^{k-1}. The relationship $upper - bound(X^k) \leq upper - bound(X^{k-1})$ always holds for average utility occupancy of any node in the AUO-Tree.*

Proof The AUOL of itemset X^k is obtained by joining the AUOL of X^{k-1} and one of its child nodes. Since $transsup_{X^k} \subseteq transsup_{X^{k-1}}$, the average of the top k values in the vector $V_{\downarrow avgocc}$ of the itemset(node) X^k is not greater than the average of the top k values of the itemset(node) X^{k-1}. Using Lemma 1, $upper - bound(X^k) \leq upper - bound(X^{k-1})$. Therefore, if $upper - bound(X^{k-1}) < min_occ$, then $upper - bound(X^k) < min_occ$. □

Pruning Strategy 2 : In the proposed AUO-Tree, if the $upper - bound$ of a tree node is less than min_occ, then all the extensions of the node can be pruned directly and none of them are needed to be explored. The $upper - bound$ of any node is denoted by $X.auoub$, [Algorithm 2 Steps 12-24].

Step 7: According to Lemma 1, Theorem 3, pruning strategy 2, $a.auoub = [0.81 + 0.81/2] = 0.81$, $a.sup = 5$. As $a.auoub > 0.37(min_occ)$, item {a} is not pruned and AUOL for itemsets {ab} and {ad} are constructed, [Algorithm 2 Steps 12-24], [Algorithm 3].

Definition 4.7 (*AUOL for itemset* $k = 2$) AUOL of an itemset X_{ab} for a 2-itemset is denoted by $X_{ab}.AUOL$, where each transaction in the list consists of $tr_d, iuo, imuo, tu$, where $X_{ab}.iuo$ for each transaction in $X_{ab}.AUOL$ is defined as,

$$E_{ab}.iuo = E_a.iuo + E_b.iuo. \tag{20}$$

and $X_{ab}.imuo$ for each transaction in AUOL is defined as,

$$E_{ab}.imuo = max(E_a.imuo, E_b.imuo). \tag{21}$$

where $X_a.Tr_d == X_b.Tr_d$ and $X_a, X_b \in IT$.

Step 8: Consider AUOL of {ab} where {ab} is the prefix. $ab.avgiuo = [(0.55 + 0.99 + 0.76)/3]/2 = 0.39$, $ab.avgituo = 0.39 * 0.41 = 0.16$, $ab.sup = 3$. As $ab.sup = 3 > 2(min_sup)$ but $ab.avgituo < 0.37(min_occ)$ itemset {ab} is not a HAUOP, [Algorithm 2 Steps 6-11].

Definition 4.8 (*AUOL for itemset* $k > 2$) AUOL of an itemset X_{ab} for an itemset where $k > 2$ is denoted by $X_{ab}.AUOL$, where each transaction consists of $tr_d, iuo, imuo, tu$, where $X_{ab}.iuo$ for each transaction in $X_{ab}.AUOL$ can be defined as,

$$E_{ab}.iuo = (E_a.iuo + E_b.iuo) - (E.iuo). \tag{22}$$

and $X_{ab}.imuo$ can be defined as,

$$E_{ab}.imuo = max(E.imuo, E_a.imuo, E_b.imuo). \tag{23}$$

where $X.Tr_d == X_a.Tr_d == X_b.Tr_d$ and $X, X_a, X_b \in IT$.

Pruning Strategy 3: If $X.AUOL$ is empty, the itemset X is pruned and none of its supersets are explored, [Algorithm 3 Steps 16-18] [14].

For example, $ce.AUOL$ is empty; therefore, itemset $\{ce\}$ is pruned and none of its supersets are explored.

Step 9: According to Lemma 1, Theorem 3, pruning strategy 2, $ab.auoub = [0.81 + 0.46/2] = 0.63$, $ab.sup = 3$. As $ab.auoub > 0.37(min_occ)$, item {ab} is not pruned and $abd.AUOL$ for itemset $\{abd\}$ is constructed, [Algorithm 2 Steps 12-19].

Step 10: Consider item(node) {b} as a prefix. $b.sup = 5$, $b.avgituo = 0.38$. As $b.sup > 2(min_sup)$ and $b.avgituo > 0.37(min_occ)$ item {b} is a HAUOP, [Algorithm 2 Steps 6-11].

Step 11: From itemset $b.AUOL$, $b.sup = 5$ and $b.auoub = [(0.81 + 0.75)/2] = 0.78$. As $b.auoub > 0.37(min_occ)$, the AUOL of itemset {bd} is constructed, [Algorithm 2 Steps 12-24].

Step 12: From itemset $bd.AUOL$, $bd.sup = 2$ and $bd.avgituo = 0.12$. As $bd.sup = 2(min_sup)$ but $bd.avgituo < 0.37(min_occ)$, itemset $\{bd\}$ is not a HAUOP, [Algorithm 2 Steps 6-11].

By using transaction database Table 4 and external utility Table 5, the proposed $HAUOPM$ algorithm mines item $\{b\}$ as high average utility occupancy pattern (HAUOP).

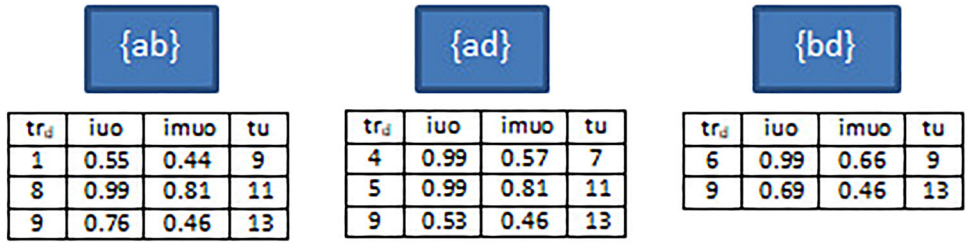
4.2 Complexity Analysis

We assume that there are m distinct items and n transactions in a database DB. The first database scan requires $O(m \times n)$ time in the worst case to calculate the $support$ values of all items it_p in IT. $O(mlogm)$ time is necessary to sort each item it_p, in ascending order of $support$, where $it_p \in IT$. $O(m \times n)$ time is necessary to scan the database DB, and to construct AUOL of each item it_p in IT. In the worst case, $O((2^m - 1) \times n)$ is required to calculate the $auoub$ value of each item it_p, or an itemset X. $O(2^m - 1)$ time is required in the worst case to traverse the search-space to mine all high average utility occupancy itemsets. Therefore, time complexity of HAUOPM in the worst case is $O((2^m - 1) \times n)$.

5 Experimental Results

In this section, experiments have been performed using two real data sets, namely retail and kosarak [27] (https://www.philippe-fournier-viger.com/spmf/index.php?link=datasets.php). Synthetic datasets, namely t5i10d10k, t6i12d10k, t7i13d10k and t8i14d10k, have also been used to test the efficiency of the proposed system. All the synthetic datasets have been generated using the spmf library [27]. To generate synthetic datasets in java, HashMap class is used to associate each item with an external utility(profit). All external

Fig. 4 AUOL of {ab}, {ad}, {bd}



{ab}

tr_d	iuo	imuo	tu
1	0.55	0.44	9
8	0.99	0.81	11
9	0.76	0.46	13

{ad}

tr_d	iuo	imuo	tu
4	0.99	0.57	7
5	0.99	0.81	11
9	0.53	0.46	13

{bd}

tr_d	iuo	imuo	tu
6	0.99	0.66	9
9	0.69	0.46	13

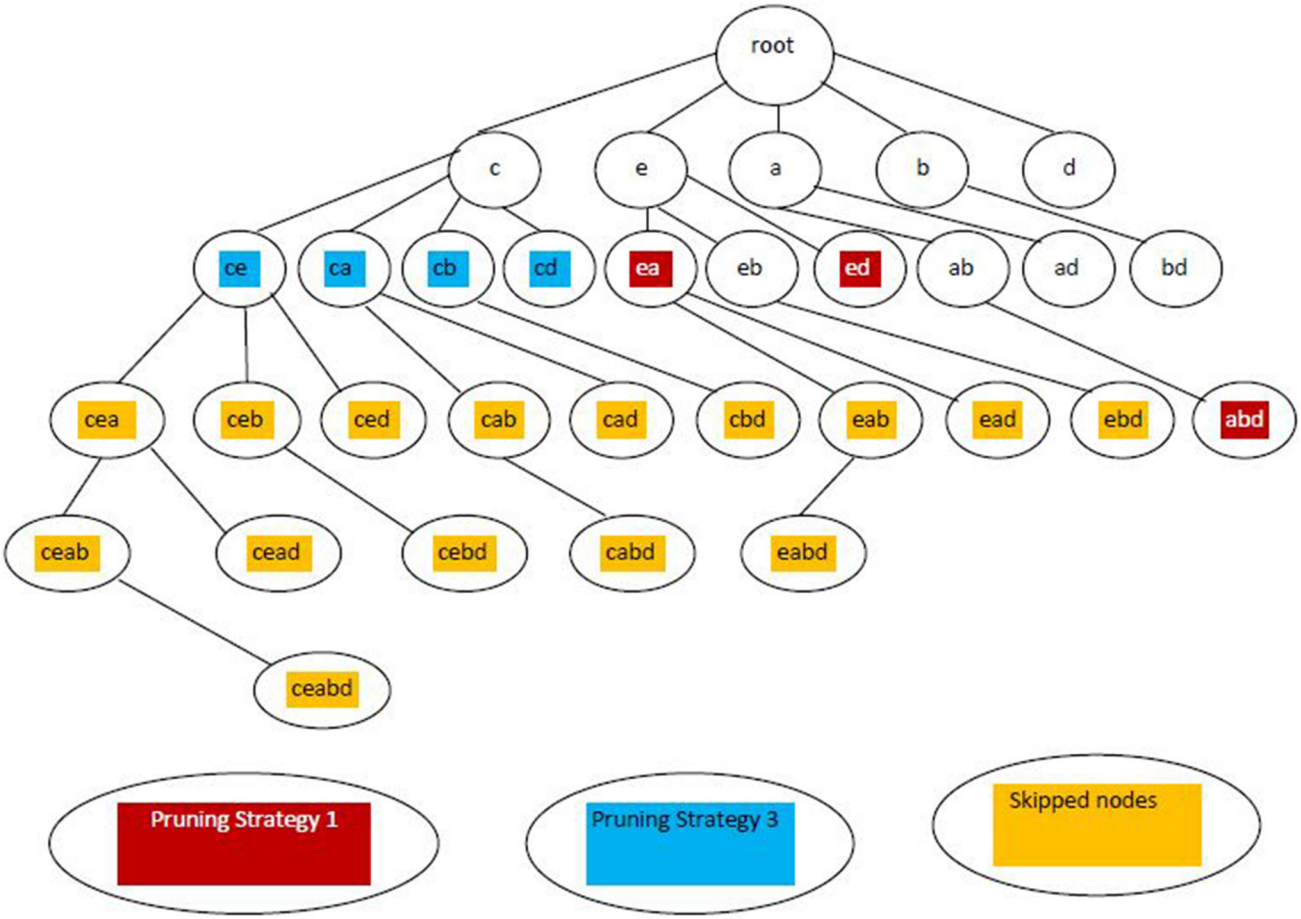


Fig. 5 Pruned-search-space-HAUOPM

utilities are randomly generated. Internal utility(quantity) for each item for every transaction is also randomly generated. The utility of each item in every transaction was obtained by taking the product of its internal and external utility values. The items and utilities are then converted into spmf [27] format for execution. A snapshot of synthetic dataset t5i10d10k is illustrated using Fig. 6. All experiments have been performed using Dell Intel Core i3 1.70 GHz with 8GB RAM and 64 bit windows 10 operating system.

The characteristics of the datasets used in experimentation is as follows:

1. ***retail***: This dataset contains 88, 162 transactions with 16, 470 distinct items. It contains an average transaction length of 10.30 items. It has a density of 0.06%.

```
752 826 834:29:8 3 18
720 752 834:46:18 10 18
685 751 801 811 852:22:3 1 4 8 6
685 751 801 811 852:25:7 8 4 1 5
661 695 716 721 727 747 938:64:7 8 4 24 5 2 14
641 721 948 980:51:16 21 6 8
641 721 948 980:49:20 12 10 7
624 654 691 702:31:14 10 6 1
615 630:17:7 10
601 856 900 938:18:8 4 2 4
599 643 938:20:6 10 4
```

Fig. 6 SyntheticDataset

2. ***kosarak***: This dataset contains 990, 002 transactions with 41, 270 distinct items. It contains an average transaction length of 8.1 items. It has a density of 0.02%.

Table 7 Number of patterns

Dataset	Symbols	Number of patterns				
		Test1	Test2	Test3	Test4	Test5
retail	min_occ	0.13	0.15	0.17	0.19	0.21
$min_sup = 800$	$HAUOPM_{baseline}$	2	2	2	1	1
	$HAUOPM_{auoub}$	2	2	2	1	1
	$HAUOPM$	2	2	2	1	1
kosarak	min_occ	0.15	0.17	0.19	0.21	0.23
$min_sup = 6000$	$HAUOPM_{baseline}$	3	3	3	2	2
	$HAUOPM_{auoub}$	3	3	3	2	2
	$HAUOPM$	3	3	3	2	2
t5i10d10k	min_occ	0.14	0.15	0.16	0.17	0.18
$min_sup = 15$	$HAUOPM_{baseline}$	5	1	1	1	1
	$HAUOPM_{auoub}$	5	1	1	1	1
	$HAUOPM$	5	1	1	1	1
t6i12d10k	min_occ	0.10	0.11	0.12	0.13	0.14
$min_sup = 15$	$HAUOPM_{baseline}$	5	5	5	4	1
	$HAUOPM_{auoub}$	5	5	5	4	1
	$HAUOPM$	5	5	5	4	1
t7i13d10k	min_occ	0.001	0.01	0.1	0.11	0.12
$min_sup = 10$	$HAUOPM_{baseline}$	1211	155	4	4	4
	$HAUOPM_{auoub}$	1211	155	4	4	4
	$HAUOPM$	1211	155	4	4	4
t8i14d10k	min_occ	0.001	0.01	0.1	0.11	0.12
$min_sup = 20$	$HAUOPM_{baseline}$	2948	288	3	3	3
	$HAUOPM_{auoub}$	2948	288	3	3	3
	$HAUOPM$	2948	288	3	3	3

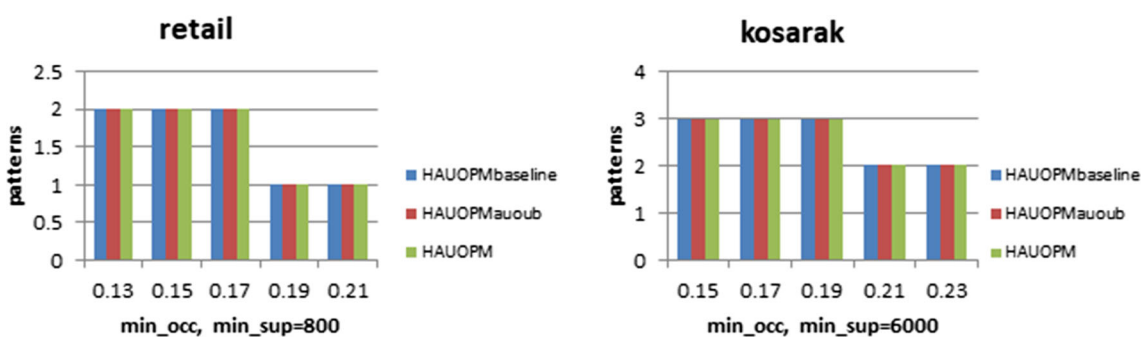


Fig. 7 Patterns-real datasets

3. ***t5i10d10k***: This dataset contains 10, 000 transactions with 10 distinct items. It contains an average transaction length of 2.98 items. It has a density of 29.8%.
4. ***t6i12k10k***: This dataset contains 10, 000 transactions with 12 distinct items. It contains an average transaction length of 3.52 items. It has a density of 29.33%.
5. ***t7i13k10k***: This dataset contains 10, 000 transactions with 13 distinct items. It contains an average transaction length of 3.98 items. It has a density of 30.61%.
6. ***t8i14k10k***: This dataset contains 10, 000 transactions with 14 distinct items. It contains an average transaction length of 4.5 items. It has a density of 32.14%.

Algorithms used to test the efficiency of the proposed system:

1. $HAUOPM_{baseline}$: This baseline algorithm uses *average utility occupancy* and *transaction utility occupancy* as measures without using any of the pruning strategies.

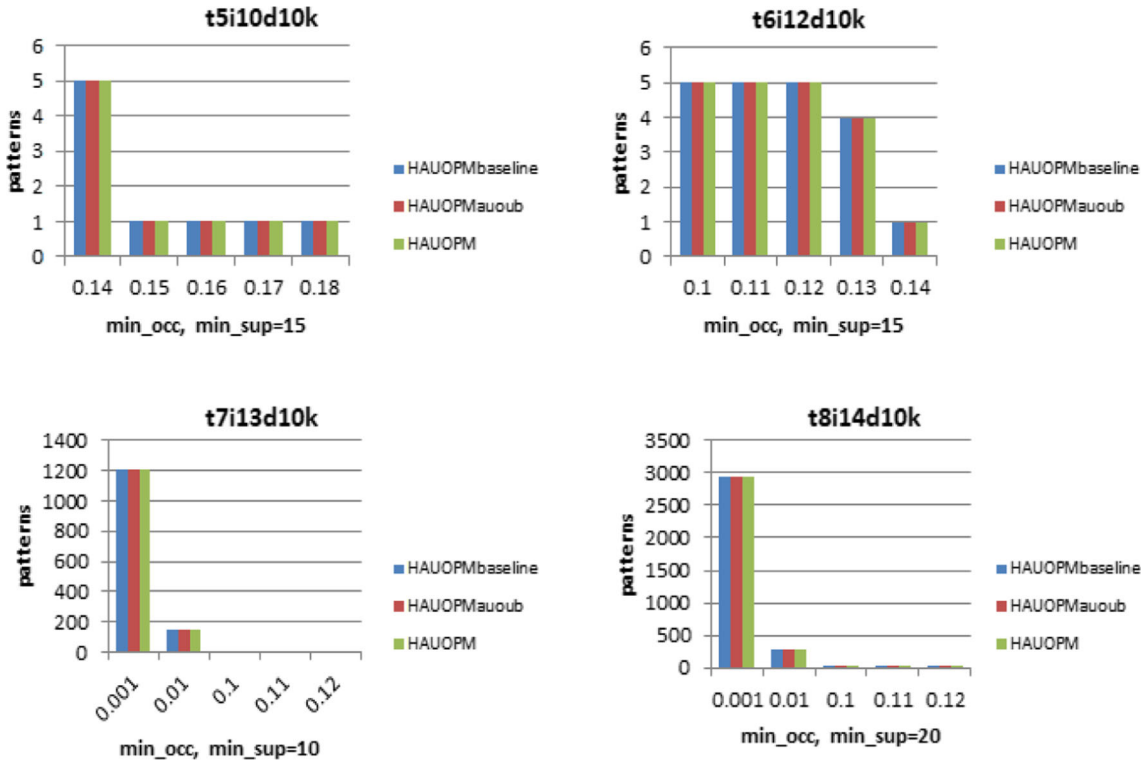


Fig. 8 Patterns-synthetic datasets

2. $HAUOPM_{auoub}$: This algorithm uses *average utility occupancy* and *transaction utility occupancy* measures along with *auoub* as a pruning strategy.
3. $HAUOPM$: This algorithm uses *average utility occupancy* and *transaction utility occupancy* measures along with all the pruning strategies 1, 2, 3 to mine high-quality patterns.

Effect of pruning strategies on runtime and memory of the proposed $HAUOPM$ algorithm has been compared with $HAUOPM_{baseline}$, $HAUOPM_{auoub}$ algorithms. The experimental results of the proposed $HAUOPM$ algorithm are not compared with $HUOPM$ [14], $OCEAN$ [12] algorithms as the proposed system uses two novel measures called *average utility occupancy* and *transaction utility occupancy*. Experiments have been performed to show the effect of pruning strategies on runtime and memory utilization of the proposed system. In all the experiments performed on various real and synthetic datasets, the min_occ threshold is increased and min_sup is kept constant. As min_occ increases the number of patterns decrease, efficiency in terms of runtime increases and memory utilization decreases.

5.1 Pattern Analysis

In this subsection, we analyse the patterns generated by the proposed $HAUOPM$ algorithm and compare it with $HAUOPM_{baseline}$, $HAUOPM_{auoub}$ algorithms. $HAUOPM$ algorithm mines HAUOPs using all the three pruning strategies 1, 2, 3. $HAUOPM_{baseline}$, $HAUOPM_{auoub}$ and $HAUOPM$ algorithms mine the same number of patterns but the number of candidates(nodes) generated by each algorithm might vary. This underlying fact shows the completeness and correctness of the proposed system. For example, while using retail dataset at $min_occ = 0.13$, $min_sup = 800$ two itemsets are mined using $HAUOPM_{baseline}$, $HAUOPM_{auoub}$ and $HAUOPM$ algorithms, whereas the number of nodes(candidates) visited is 3636, 3628, 3617, respectively. Similarly, while using $t6i1210k$, at $min_occ = 0.11$ and $min_sup = 15$, all the three algorithms derive 5 itemsets whereas the number of nodes(candidates) visited are 2076, 1791 and 1640, respectively. The following observations can be made form Table 7, Fig. 7 and figure 8:

Table 8 HAUOPs-retail

HAUOP	Average transaction utility occupancy	Support
49	0.222	42135.0
40	0.183	50675.0

1. The $HAUOPM$ algorithm is capable of mining patterns that are both concise and of high quality. Specifically, the

Table 9 Runtime analysis

Dataset	Symbols	Runtimes using various thresholds				
		Test1	Test2	Test3	Test4	Test5
retail	min_occ	0.13	0.15	0.17	0.19	0.21
$min_sup = 800$	$HAUOPM_{baseline}$	3839	3828	3859	3816	4240
	$HAUOPM_{auoub}$	3762	3701	3731	3745	4088
	$HAUOPM$	3740	3696	3633	3559	3598
kosarak	min_occ	0.15	0.17	0.19	0.21	0.23
$min_sup = 6000$	$HAUOPM_{baseline}$	43438	42016	41236	41098	41059
	$HAUOPM_{auoub}$	39267	36284	28529	27532	26753
	$HAUOPM$	37489	35627	28240	28080	26051
t5i10d10k	min_occ	0.14	0.15	0.16	0.17	0.18
$min_sup = 15$	$HAUOPM_{baseline}$	1101	926	914	860	840
	$HAUOPM_{auoub}$	972	908	872	823	806
	$HAUOPM$	909	887	844	839	778
t6i12d10k	min_occ	0.10	0.11	0.12	0.13	0.14
$min_sup = 15$	$HAUOPM_{baseline}$	1617	1353	1172	1106	1082
	$HAUOPM_{auoub}$	1541	1225	1166	1026	1000
	$HAUOPM$	1319	1156	1121	1006	864
t7i13d10k	min_occ	0.001	0.01	0.1	0.11	0.12
$min_sup = 10$	$HAUOPM_{baseline}$	1636	1567	1395	1299	1250
	$HAUOPM_{auoub}$	1610	1540	1339	1259	1251
	$HAUOPM$	1431	1390	1296	1190	1174
t8i14d10k	min_occ	0.001	0.01	0.1	0.11	0.12
$min_sup = 20$	$HAUOPM_{baseline}$	3008	2491	2227	2183	2064
	$HAUOPM_{auoub}$	2231	2130	2052	1970	1833
	$HAUOPM$	1986	1841	1626	1621	1579

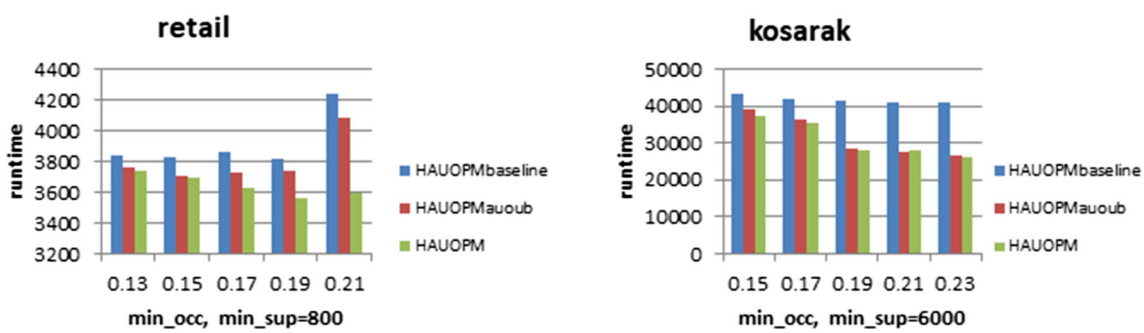


Fig. 9 Runtime-real datasets

patterns extracted using this algorithm are characterized by both high occupancy and high utility.

2. By employing the $HAUOPM$ algorithm, it is observed that increasing the value of the minimum occupancy threshold (min_occ) results in a reduction in the number of patterns that are mined from the dataset.
3. $HAUOPM_{baseline}$, $HAUOPM_{auoub}$ and $HAUOPM$ algorithms derive the same number patterns but the number of candidate patterns (nodes) generated by each algorithm may differ depending on the specific pruning strategies that are employed.

Table 8 shows HAUOPs derived for retail dataset at $min_occ = 0.13$ and $min_sup = 800$.

5.2 Efficiency Evaluation

To evaluate the efficiency of the proposed $HAUOPM$ algorithm, a runtime evaluation and comparison is performed on $HAUOPM_{baseline}$ and $HAUOPM_{auoub}$ algorithms. The results in terms of varying values of min_occ and constant min_sup can be observed according to Table 9, Figs. 9 and 10. For example, when retail dataset was used at $min_occ =$

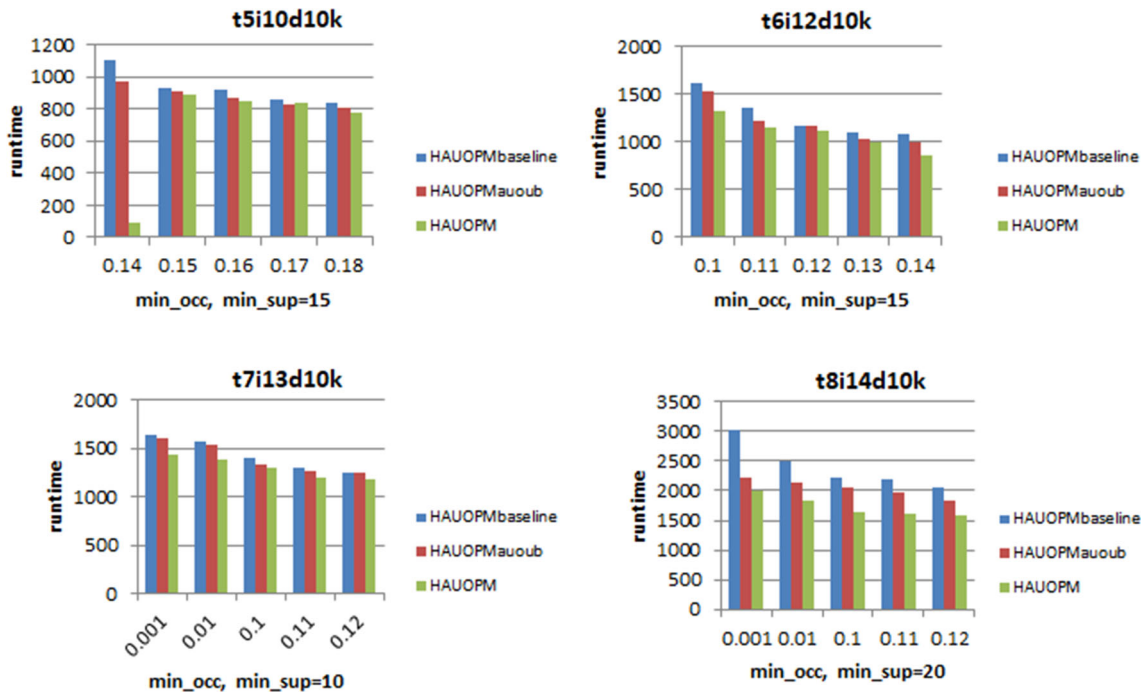


Fig. 10 Runtime-synthetic datasets

Table 10 Memory utilized

Dataset	Symbols	Memory				
		Test1	Test2	Test3	Test4	Test5
retail	min_occ	0.13	0.15	0.17	0.19	0.21
$min_sup = 800$	$HAUOPM_{baseline}$	94.17	94.17	93.66	93.65	94.58
	$HAUOPM_{auoub}$	93.65	94.11	93.65	93.58	94.51
	$HAUOPM$	93.65	93.65	93.45	93.44	93.44
kosarak	min_occ	0.15	0.17	0.19	0.21	0.23
$min_sup = 6000$	$HAUOPM_{baseline}$	737.90	696.5	683.67	631.48	622.90
	$HAUOPM_{auoub}$	711.99	727.42	696.50	682.18	633.64
	$HAUOPM$	564.60	555.5	613.09	595.56	580.45
t5i10d10k	min_occ	0.14	0.15	0.16	0.17	0.18
$min_sup = 15$	$HAUOPM_{baseline}$	24.73	24.26	24.23	24.21	24.22
	$HAUOPM_{auoub}$	24.25	24.24	24.22	23.22	21.23
	$HAUOPM$	24.24	24.23	24.24	23.23	21.75
t6i12d10k	min_occ	0.10	0.11	0.12	0.13	0.14
$min_sup = 15$	$HAUOPM_{baseline}$	41.69	40.66	40.13	40.17	39.68
	$HAUOPM_{auoub}$	41.66	40.69	40.16	40.18	39.67
	$HAUOPM$	41.71	40.67	40.15	40.16	39.67
t7i13d10k	min_occ	0.001	0.01	0.1	0.11	0.12
$min_sup = 10$	$HAUOPM_{baseline}$	67.63	67.13	67.08	60.10	57.62
	$HAUOPM_{auoub}$	67.63	67.11	67.13	54.61	54.12
	$HAUOPM$	57.64	57.62	57.63	54.12	54.08
t8i14d10k	min_occ	0.001	0.01	0.1	0.11	0.12
$min_sup = 20$	$HAUOPM_{baseline}$	75.54	75.52	75.52	57.53	57.60
	$HAUOPM_{auoub}$	75.54	75.54	75.53	58.53	57.54
	$HAUOPM$	75.54	75.52	75.51	58.54	57.52

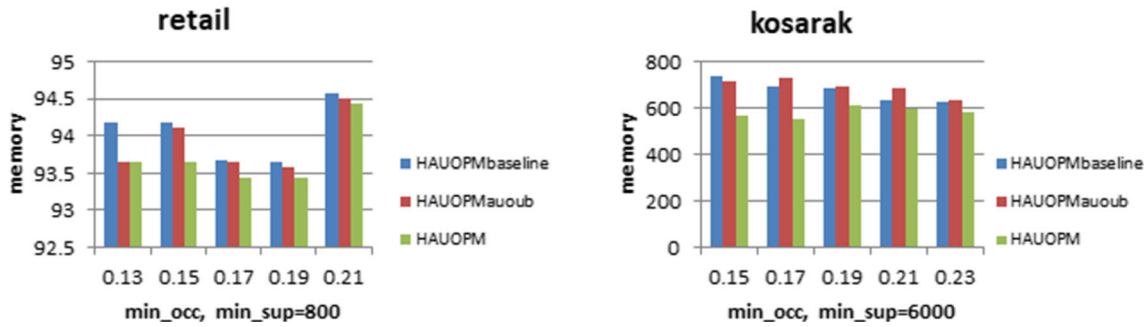


Fig. 11 Memory-real datasets

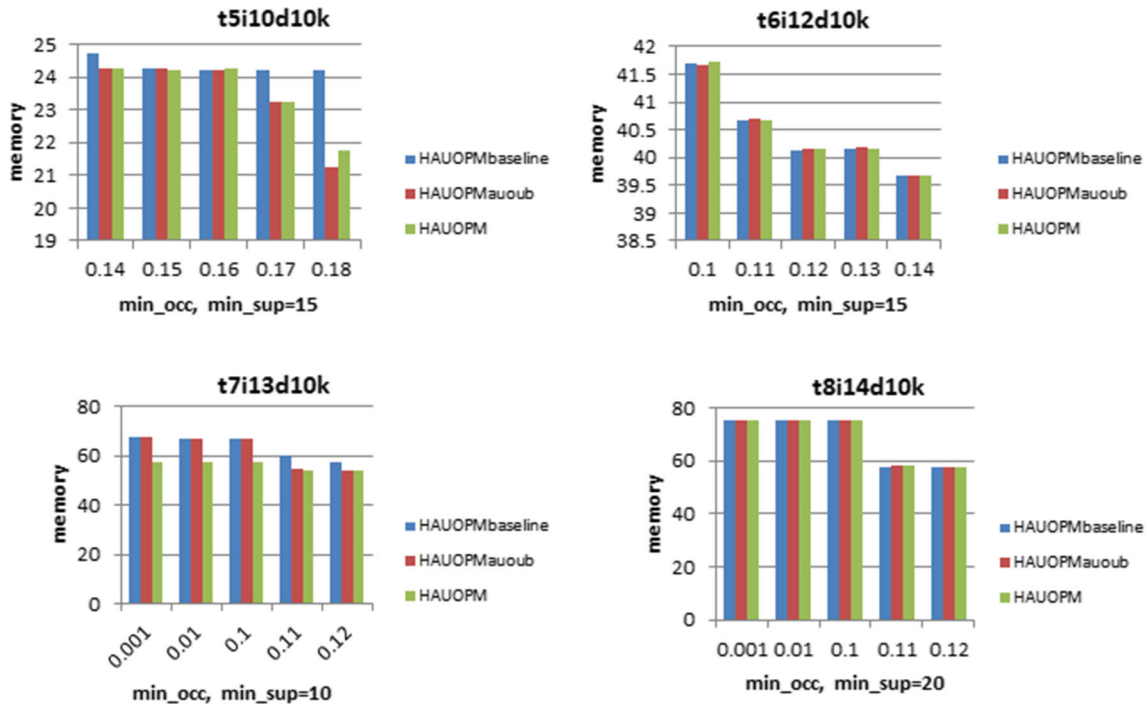


Fig. 12 Memory-synthetic datasets

0.13 and $min_sup = 800$, the proposed $HAUOPM$ algorithm is 99 ms faster than the $HAUOPM_{baseline}$ algorithm. As the min_occ is increased from 0.13 to 0.21, the efficiency of the proposed $HAUOPM$ algorithm increases in terms of execution time. For example at $min_occ = 0.19$, the proposed $HAUOPM$ algorithm is faster than the $HAUOPM_{baseline}$ algorithm by 257 ms. Similarly, when retail dataset was used at $min_occ = 0.13$ and $min_sup = 800$, the $HAUOPM$ algorithm was just 22 ms faster than the $HAUOPM_{auoub}$ algorithm. As $HAUOPM_{auoub}$ uses just one pruning strategy(auoub) 2 when compared to three pruning strategies used by $HAUOPM$ algorithm, $HAUOPM$ algorithm is much more effective in terms of pruning candidate(nodes) itemsets, thus increasing the efficiency of the algorithm in terms of runtime. Similarly, when synthetic dataset $t6i12d10k$ was used at $min_occ = 0.14$ and $min_sup = 15$, the proposed $HAUOPM$ algorithm is 218 ms faster than the $HAUOPM_{baseline}$ algorithm. Based on Table 9, Figs. 9 and 10 some important findings are:

1. $HAUOPM$ algorithm uses all the three pruning strategies 1, 2, 3, to reduce the search space in terms of the number of nodes. By visiting fewer nodes during the mining process, the algorithm's efficiency is increased, resulting in reduced runtime.
2. Using $HAUOPM$, as min_occ threshold value increases the runtime of the proposed $HAUOPM$ algorithm decreases.
3. Execution time of $HAUOPM$ algorithm is much less than that of the $HAUOPM_{auoub}$ algorithm as $HAUOPM_{auoub}$, uses just one pruning strategy(auoub)2

Table 11 Effect of pruning strategies

Dataset	Symbols	Candidates				
		Test1	Test2	Test3	Test4	Test5
retail	min_occ	0.13	0.15	0.17	0.19	0.21
$min_sup = 800$	$HAUOPM_{baseline}$	3636	3636	3636	3636	3636
	$HAUOPM_{auoub}$	3628	3609	3589	3584	3573
	$HAUOPM$	3617	3602	3583	3580	3573
kosarak	min_occ	0.15	0.17	0.19	0.21	0.23
$min_sup = 6000$	$HAUOPM_{baseline}$	7782	7782	7782	7782	7782
	$HAUOPM_{auoub}$	6194	5435	4463	3818	3132
	$HAUOPM$	6194	5435	4463	3818	3132
t5i10d10k	min_occ	0.14	0.15	0.16	0.17	0.18
$min_sup = 15$	$HAUOPM_{baseline}$	634	634	634	634	634
	$HAUOPM_{auoub}$	608	593	563	497	298
	$HAUOPM$	602	587	557	491	298
t6i12d10k	min_occ	0.10	0.11	0.12	0.13	0.14
$min_sup = 15$	$HAUOPM_{baseline}$	2076	2076	2076	2076	2076
	$HAUOPM_{auoub}$	2076	1791	1769	1746	1652
	$HAUOPM$	1882	1640	1618	1596	1507
t7i13d10k	min_occ	0.001	0.01	0.1	0.11	0.12
$min_sup = 10$	$HAUOPM_{baseline}$	4282	4282	4282	4282	4282
	$HAUOPM_{auoub}$	4282	4282	4282	2015	2002
	$HAUOPM$	4119	4119	4119	1990	1977
t8i14d10k	min_occ	0.001	0.01	0.1	0.11	0.12
$min_sup = 20$	$HAUOPM_{baseline}$	6772	6772	6772	6772	6772
	$HAUOPM_{auoub}$	6772	6772	6772	6772	6772
	$HAUOPM$	6534	6534	6534	6534	6534

in comparison to the three pruning strategies 1, 2, 3 used by the $HAUOPM$ algorithm.

5.3 Memory Utilization

$HAUOPM$ algorithm utilizes less amount of memory when compared to $HAUOPM_{baseline}$ and $HAUOPM_{auoub}$ algorithms. Pruning strategies 1, 2, 3 prune unpromising nodes from the search space. As the number of nodes decreases, the number of average utility occupancy lists (AUOLs) required to be stored in the memory also decreases. From Table 10, Figs. 11 and 12 when kosarak dataset was used at $min_occ = 0.15$ and $min_sup = 6000$, memory utilization of $HAUOPM$ algorithm is 173.3 MB less than $HAUOPM_{baseline}$ algorithm. Similarly when synthetic dataset $t7i13d10k$ was used at $min_occ = 0.001$ and $min_sup = 10$, the memory utilization of $HAUOPM$ algorithm is 9.99 MB less than the $HAUOPM_{baseline}$ algorithm.

5.4 Effect of Pruning Strategies

In this subsection, we access the effectiveness of pruning strategies by comparing the number of nodes visited in the AUO-Tree. Based on Table 11, Figs. 13 and 14, it can be observed that when retail dataset was used at $min_occ = 0.21$ and $min_sup = 800$, $HAUOPM$ algorithm visits 63 nodes less than $HAUOPM_{baseline}$ algorithm. Similarly, when synthetic dataset $t7i13d10k$ was used at $min_occ = 0.12$ and $min_sup = 10$, $HAUOPM$ algorithm visits 2305 nodes less than $HAUOPM_{baseline}$ algorithm. The following key observations can be made based on Table 11, Figs. 13 and 14:

1. The $HAUOPM$ algorithm prunes the search space in terms of the number of nodes, making it faster as the minimum occupancy threshold (min_occ) increases.
2. Due to the significant pruning of the search space in $HAUOPM$, the size of the AUO-Tree is reduced. This reduction in size leads to faster processing times as compared to the processing time required for the complete search-space
3. A smaller search space due to pruning strategies results in faster runtime and lesser utilization of memory.

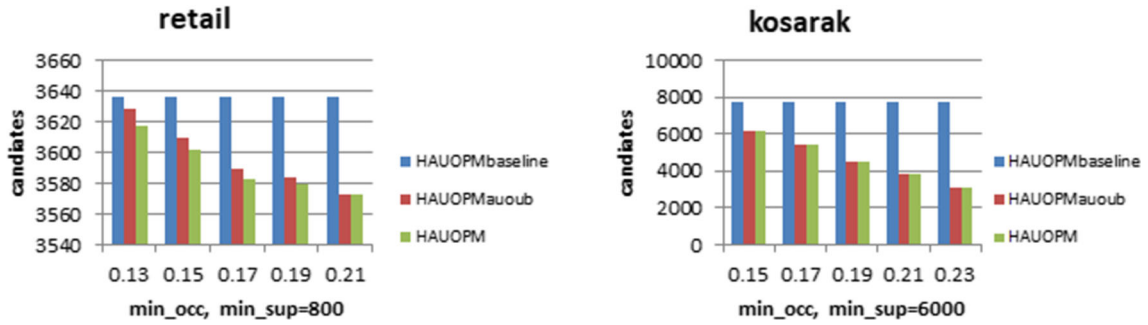


Fig. 13 Candidates-real datasets

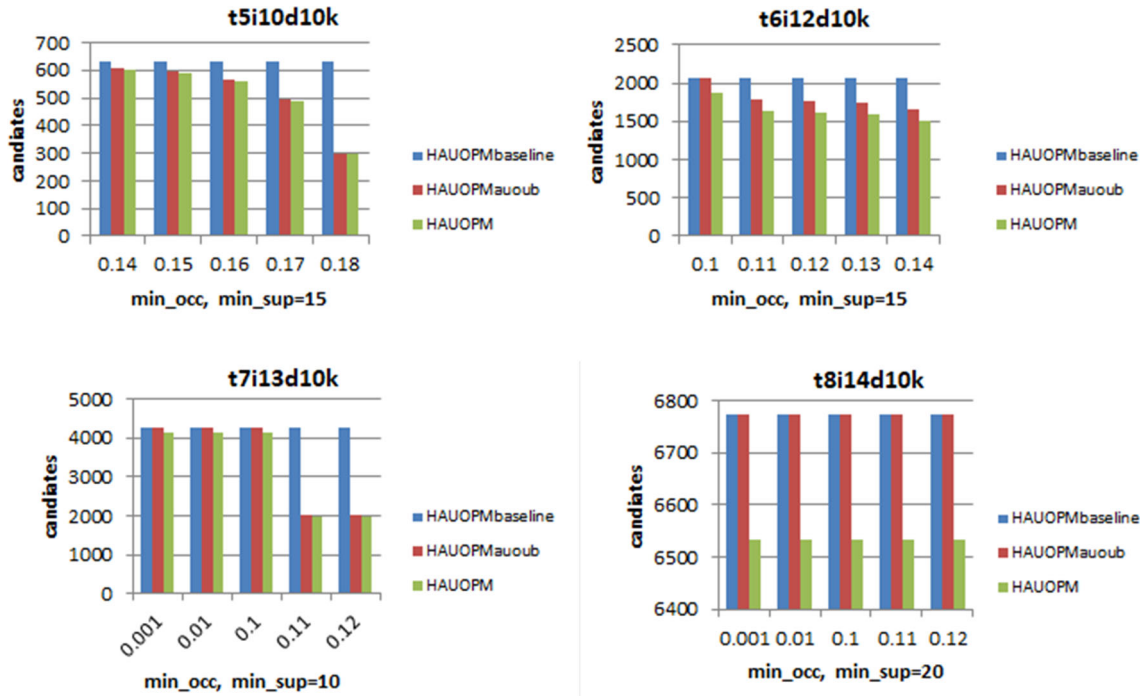


Fig. 14 Candidates-synthetic datasets

Table 12 HAUOPM-NIFTY50 (bank)

Dataset	Algorithm	HAUOPM-NIFTY-50 (bank)				
		Test1	Test2	Test3	Test4	Test5
NIFTY-50(bank)($min_sup = 100$)	min_occ	0.05	0.04	0.03	0.02	0.01
	Stocksets	3476	3725	3874	3966	4016
	Candidates	4083	4083	4083	4083	4083
	Runtime (ms)	2420	2289	2295	2254	2568
	Memory (MB)	76.0	75.98	76.0	76.0	76.0

5.5 Generalization of the Proposed HAUOPM Algorithm

The HAUOPM algorithm is not limited to a particular type of item and can be utilized for any item that has a weight or profit value associated with it. This feature enables HAUOPM to be applied in various fields beyond retail and e-commerce, such as health care and finance. One potential application of the HAUOPM algorithm in the financial domain is demonstrated through the analysis of the NIFTY-50(bank) stock dataset. To accomplish this, we organize the stocks into sets, which we refer to as "stocksets," and convert the raw data from a 494-day trading period into the SPMF [27] format for efficient mining of high average utility occu-

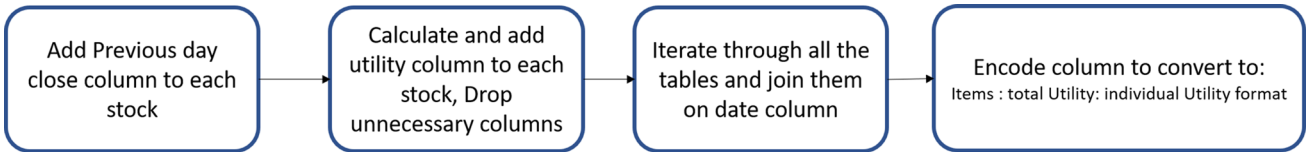


Fig. 15 Data preprocessing

Fig. 16 Previous close

Date	Open	High	Low	Close	Adj Close	Volume	Prev Close
1995-12-26	72.819626	72.819626	72.819626	72.819626	53.472786	0.0	72.819626
1995-12-27	72.819626	72.819626	72.819626	72.819626	53.472786	0.0	72.819626
1995-12-28	72.819626	72.819626	72.819626	72.819626	53.472786	0.0	72.819626
1995-12-29	72.819626	72.819626	72.819626	72.819626	53.472786	0.0	72.819626
1996-01-01	72.436363	72.819626	72.340553	72.819626	53.472786	40703.0	72.819626
1996-01-02	72.819626	73.566986	72.053108	72.398041	53.163197	100975.0	72.819626

Date	Utility
2002-07-02	0.3454
2002-07-03	-0.0257
2002-07-04	1.0225
2002-07-05	0.0658
2002-07-08	-0.9857

Fig. 17 Utility column

pancy stocksets. Each stock in the dataset is associated with a corresponding profit or loss value, which enables us to leverage the HAUOPM algorithm for this analysis. These stocksets can be used by investors and traders to recognize *high-utility occupancy stocksets*. We gathered historical data for all stocks in the NIFTY-50(bank) by utilizing Yahoo Finance's website (https://finance.yahoo.com/). The dataset we obtained includes several columns of information for each stock, with a specific focus on the percentage difference between the previous day's closing price and the current day's closing price for each stock on a given day. This approach enabled us to collect comprehensive data for all the stocks in the NIFTY-50(bank) dataset.

Utility = (todays close value − previous day close value) / (previous day close value)

Steps to convert raw data into spmf [27] format:

1. Add closing value of previous day to each row of each file of stock, Fig. 16.
2. Add utility column, Fig. 17.
3. Join utility columns based on date, Fig. 18.
4. Encode and convert to.txt format which is used as an input to the proposed HAUOPM algorithm. For example, snapshot of *NIFTY-50*(bank) stocks converted into spmf [27] format is shown in Fig. 19. Each transaction has the following format:
   Items:Transaction Utility: Individual Utility

The proposed HAUOPM algorithm applied on *NIFTY-50*(bank) dataset yields the following results:

Date	TATASTEEL_Utility_percent	ADANIPORTS_Utility_percent	APOLLOHOSP_Utility_percent	ASIANPAINT_Utility_percent	AXISBANK_Utility_percent	BAJAJ-AUTO_Utility_percent
2018-04-27	-2.0062	-1.3473	0.4179	-2.2411	-9.2051	-2.156
2018-04-30	-0.8732	-1.2051	-1.7344	-1.7011	4.0616	-0.1849
2018-05-02	3.4373	1.5222	1.0037	-1.7142	-1.2469	-0.4521
2018-05-03	-1.906	1.0222	0.9953	1.2722	-1.8425	-0.3119
2018-05-04	0.8028	-2.8338	-3.002	1.3507	2.4559	2.909
2018-05-07	-2.7034	-1.0165	1.8613	-2.0833	-3.0367	-0.9226
2018-05-08	0.0168	-0.097	0.8604	0.4402	-1.1472	0.7907
2018-05-09	-1.1403	-0.1332	2.4356	-1.0455	-1.1526	0.7001
2018-05-10	1.4922	0.6169	2.0421	0.3967	0.155	1.5494
2018-05-11	-2.2048	-0.4625	-2.0261	-5.6459	-1.0317	0.4262
2018-05-14	-0.5105	0.6663	2.0289	-0.8473	0.253	-0.5861
2018-05-15	-2.5559	0.1952	-1.7681	-1.3335	0.3986	0.2931
2018-05-16	0.7509	1.4787	1.1855	-0.6884	-0.0546	0.4161
2018-05-17	1.7626	1.29	1.1317	1.0765	1.9182	-0.1263
2018-05-18	3.0968	2.1111	0.113	0.084	1.2142	0.9927

Fig. 18 Stock utility columns

```
1 2 3 4 5 6 7 8 9 10 11 12:18.129:0.159 6.315 1.209 -0.304 2.481 0.696 1.044 2.85 2.651 -0.295 0 0.125
1 2 3 4 5 6 7 8 9 10 11 12:13.508:0.554 -1.491 -1.444 -0.152 0.553 -0.372 1.759 4.282 0.076 0.543 -0.985 1.297
1 2 3 4 5 6 7 8 9 10 11 12:28.004:-0.175 2.575 2.475 0.992 4.058 -0.303 -1.024 10.628 3.22 -0.914 0.71 0.93
```

Fig. 19 Nifty50 (bank)-SPMF-FORMAT

6 Conclusion and Future Work

Our paper presents a novel and efficient approach to mine all high average utility occupancy patterns (HAUOPs). To address the limitations of the utility occupancy measure, we have introduced a new measure called *average utility occupancy*, which provides a fair representation of the occupancy of each item in an itemset. Additionally, we have proposed a new measure called *transaction utility occupancy* to handle items with low utility occupancy but high occupancy and vice versa. To efficiently enumerate the search space, we have utilized a novel *AUO-Tree* structure, along with an effective pruning strategy called *average utility occupancy upper-bound (auoub)* to discard unpromising itemsets, thereby reducing runtime and memory allocation. Furthermore, we have introduced a modified *average utility occupancy list (AUOL)* data structure to store information about itemsets. Our experimental results, using various real and synthetic datasets, show that our proposed HAUOPM algorithm outperforms the baseline $HAUOPM$ algorithms, $HAUOPM_baseline$ and $HAUOPM_auoub$, in terms of runtime and memory allocation. For future work, we aim to develop new upper-bounds to make the algorithm even more efficient and extend it to handle uncertain and dynamic databases.

References

1. Chen, M.-S.; Han, J.; Yu, P.S.: Data mining: an overview from a database perspective. IEEE Trans. Knowl. Data Eng. **8**(6), 866–883 (1996)
2. Gan, W.; Lin, J.C.-W.; Fournier-Viger, P.; Chao, H.-C.; Zhan, J.: Mining of frequent patterns with multiple minimum supports. Eng. Appl. Artif. Intell. **60**, 83–96 (2017)
3. Wang, Z.; Zhu, Y.; Wang, D.; Han, Z.: Fedfpm: a unified federated analytics framework for collaborative frequent pattern mining. In: IEEE INFOCOM 2022-IEEE Conference on Computer Communications, pp. 61–70. IEEE (2022)
4. Liu, M.; Qu, J.: Mining high utility itemsets without candidate generation. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management, pp. 55–64 (2012)
5. Fournier-Viger, P.: Fhn: efficient mining of high-utility itemsets with negative unit profits. In: International Conference on Advanced Data Mining and Applications, pp. 16–29. Springer (2014)
6. Han, M.; Gao, Z.; Li, A.; Liu, S.; Mu, D.: An overview of high utility itemsets mining methods based on intelligent optimization algorithms. Knowl. Inf. Syst. 1–40 (2022)
7. Fang, W.; Zhang, Q.; Lu, H.; Lin, J.C.-W.: High-utility itemsets mining based on binary particle swarm optimization with multiple adjustment strategies. Appl. Soft Comput. 109073 (2022)
8. Wu, P.; Niu, X.; Fournier-Viger, P.; Huang, C.; Wang, B.: Ubp-miner: an efficient bit based high utility itemset mining algorithm. Knowl. Based Syst. **248**, 108865 (2022)
9. Sethi, K.K.; Ramesh, D.; Trivedi, M.C.: A spark-based high utility itemset mining with multiple external utilities. Clust. Comput. **25**(2), 889–909 (2022)
10. Gan, W.; Lin, J.C.-W.; Chao, H.-C.; Fournier-Viger, P.; Wang, X.; Yu, P.S.: Utility-driven mining of trend information for intelligent system. arXiv:1912.11666 (2019)
11. Tang, L.; Zhang, L.; Luo, P.; Wang, M.: Incorporating occupancy into frequent pattern mining for high quality pattern recommendation. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management, pp. 75–84 (2012)
12. Shen, B.; Wen, Z.; Zhao, Y.; Zhou, D.; Zheng, W.: Ocean: Fast discovery of high utility occupancy itemsets. In: Pacific-Asia Conference on Knowledge Discovery and Data Mining, pp. 354–365. Springer (2016)
13. Lin, J.C.-W., et al.: An efficient algorithm to mine high average-utility itemsets. Adv. Eng. Inform. **30**(2), 233–243 (2016)
14. Gan, W.; Lin, J.C.-W.; Fournier-Viger, P.; Chao, H.-C.; Philip, S.Y.: Huopm: high-utility occupancy pattern mining. IEEE Trans. Cybern. **50**(3), 1195–1208 (2019)
15. Agrawal, R.; Srikant, R., et al.: Fast algorithms for mining association rules. In: Proceedings of the 20th International Conference on Very Large Data Bases, VLDB, vol. 1215, pp. 487–499 (1994)
16. Han, J.; Pei, J.; Yin, Y.: Mining frequent patterns without candidate generation. ACM SIGMOD Rec. **29**(2), 1–12 (2000)
17. Tseng, V.S.; Shie, B.-E.; Wu, C.-W.; Philip, S.Y.: Efficient algorithms for mining high utility itemsets from transactional databases. IEEE Trans. Knowl. Data Eng. **25**(8), 1772–1786 (2012)
18. Zhang, L.; Luo, P.; Tang, L.; Chen, E.; Liu, Q.; Wang, M.; Xiong, H.: Occupancy-based frequent pattern mining. ACM Trans. Knowl. Discov. Data TKDD **10**(2), 1–33 (2015)
19. Chen, C.-M.; Chen, L.; Gan, W.; Qiu, L.; Ding, W.: Discovering high utility-occupancy patterns from uncertain data. Inf. Sci. **546**, 1208–1229 (2021)
20. Ryu, T.; Yun, U.; Lee, C.; Lin, J.C.-W.; Pedrycz, W.: Occupancy-based utility pattern mining in dynamic environments of intelligent systems. Int. J. Intell. Syst. **37**(9), 5477–5507 (2022)
21. Kim, H.; Ryu, T.; Lee, C.; Kim, H.; Truong, T.; Fournier-Viger, P.; Pedrycz, W.; Yun, U.: Mining high occupancy patterns to analyze incremental data in intelligent systems. ISA Trans. **131**, 460–475 (2022)
22. Nguyen, L.T.; Mai, T.; Pham, G.-H.; Yun, U.; Vo, B.: An efficient method for mining high occupancy itemsets based on equivalence class and early pruning. Knowl. Based Syst. **267**, 110441 (2023)
23. Datta, S.; Mali, K.; Ghosh, U.: High occupancy itemset mining with consideration of transaction occupancy. Arab. J. Sci. Eng. **47**(2), 2061–2075 (2022)
24. Kumar, M.J.K., Rana, D.: High average utility itemset mining: a survey. In: ICCIDE, pp. 347 (2020)
25. Yao, H.; Hamilton, H.J.; Butz, C.J.: A foundational approach to mining itemset utilities from databases. In: Proceedings of the 2004 SIAM International Conference on Data Mining, pp. 482–486. SIAM (2004)
26. Rymon, R.: Search through systematic set enumeration (1992)
27. Fournier-Viger, P.; Gomariz, A.; Gueniche, T.; Soltani, A.; Wu, C.-W.; Tseng, V.S.: Spmf: a java open-source pattern mining library. J. Mach. Learn. Res. **15**(1), 3389–3393 (2014)

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.